Лабораторная работа №14

Бессонов Андрей Максимович

Содержание

1 Цель работы **5**

2 Теоретическое введение **6**
2.1 Программирование в bash 6
2.2 Семафоры в UNIX 6
2.3 Страницы справочной системы 6

3 Выполнение лабораторной работы **7**
3.1 Подготовка рабочего каталога 7
3.2 Задание 1. Упрощённый механизм семафоров 7
3.3 Задание 2. Реализация команды `man` 9
3.4 Задание 3. Генерация случайной последовательности букв латинского алфавита 12

4 Выводы **15**

5 Контрольные вопросы **16**
5.1 Найдите синтаксическую ошибку в следующей строке: 16
5.2 Как объединить (конкатенация) несколько строк в одну? 16
5.3 Найдите информацию об утилите `seq`. Какими иными способами можно реализовать её функционал при программировании на bash? 17
5.4 Какой результат даст вычисление выражения `$((10/3))`? 17
5.5 Укажите кратко основные отличия командной оболочки `zsh` от `bash`. 17
5.6 Проверьте, верен ли синтаксис данной конструкции: 18
5.7 Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки? 18

Список литературы **20**

Список иллюстраций

3.1 Создание каталога lab14 7
3.2 Код task1.sh 8
3.3 Выполнение task1.sh 9
3.4 Код task2.sh 11
3.5 Выполнение task2.sh для несуществующей команды 11
3.6 Код task3.sh 13
3.7 Выполнение task3.sh с разной длиной 14

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Теоретическое введение

2.1 Программирование в bash

Командная оболочка bash (Bourne Again SHell) предоставляет возможности для написания скриптов: переменные, условные операторы (`if`, `case`), циклы (`for`, `while`, `until`), функции, обработка аргументов командной строки. Встроенная переменная `$RANDOM` генерирует псевдослучайные целые числа от 0 до 32767.

2.2 Семафоры в UNIX

Для синхронизации процессов можно использовать файловые блокировки. Утилита `flock` позволяет захватить эксклюзивную блокировку на файл с заданным таймаутом, что удобно для реализации бинарного семафора.

2.3 Страницы справочной системы

Справка по командам хранится в сжатых файлах в каталоге `/usr/share/man/man1/`. Для просмотра используются `zless`, `bzless`, `xzless` в зависимости от типа сжатия.

3 Выполнение лабораторной работы

В ходе работы были последовательно выполнены все задания.

3.1 Подготовка рабочего каталога

Создан каталог ~/lab14 для размещения скриптов (рис. 1).

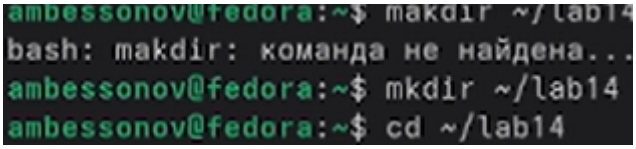

Рисунок 3.1: Создание каталога lab14

3.2 Задание 1. Упрощённый механизм семафоров

3.2.1 Листинг скрипта task1.sh

```
#!/bin/bash


LOCKFILE="/tmp/semaphore.lock"
t1=${1:-20}   # время ожидания (по умолчанию 20 сек)
t2=${2:-5}    # время использования (по умолчанию 5 сек)


if (( t2 >= t1 )); then
```

```
    echo "Ошибка: время использования (t2=$t2) должно быть меньше времени ожидания (t
    exit 1
fi

echo "Процесс $$: пытается захватить ресурс (ожидание до $t1 сек.)"

flock -w "$t1" "$LOCKFILE" -c "echo 'Процесс $$: ресурс захвачен на $t2 сек.'; sleep

if [ $? -ne 0 ]; then
    echo "Процесс $$: не удалось захватить ресурс за $t1 сек. (таймаут)"
    exit 1
fi
```

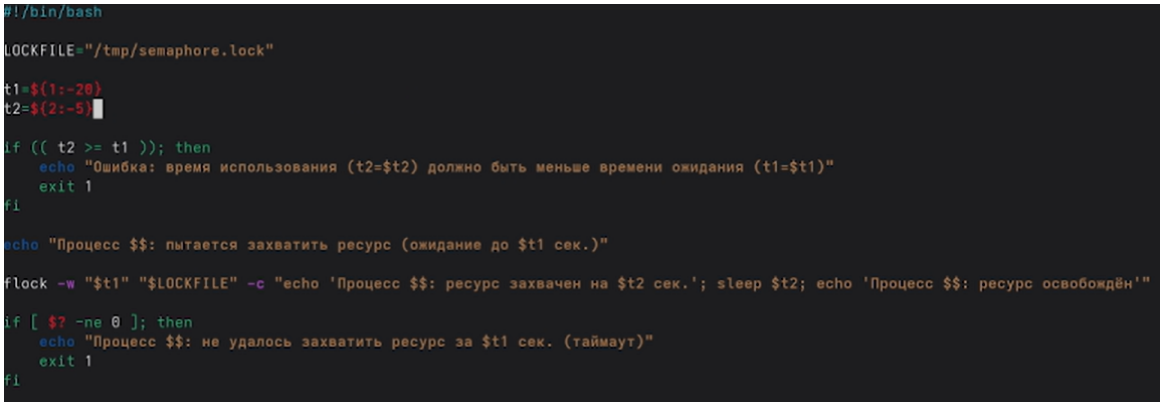

Рисунок 3.2: Код task1.sh

3.2.2 Проверка работы

Скрипт был запущен в двух терминалах. В первом терминале (привилегированный режим) – в foreground, во втором – в фоне с перенаправлением вывода в первый терминал. На рис. 3 показан результат корректной работы: процессы поочерёдно захватывают и освобождают ресурс.

```
ambessonov@fedora:~/lab14$ nano task1.sh
ambessonov@fedora:~/lab14$ chmod +x task1.sh
ambessonov@fedora:~/lab14$ ./task1.sh
Процесс 7339: пытается захватить ресурс (ожидание до 20 сек.)
Процесс 7339: ресурс захвачен на 5 сек.
Процесс 7339: ресурс освобождён
ambessonov@fedora:~/lab14$ ./task1.sh 30 10
Процесс 7346: пытается захватить ресурс (ожидание до 30 сек.)
Процесс 7346: ресурс захвачен на 10 сек.
Процесс 7346: ресурс освобождён
ambessonov@fedora:~/lab14$
```

Рисунок 3.3: Выполнение task1.sh

Для взаимодействия трёх и более процессов дополнительных изменений не требуется – `flock` автоматически ставит процессы в очередь.

3.3 Задание 2. Реализация команды man

3.3.1 Листинг скрипта task2.sh

Скрипт принимает имя команды как аргумент, ищет сжатую man-страницу в `/usr/share/man/man1/` и отображает её через `zless` (или `bzless`/`xzless` в зависимости от расширения).

```
#!/bin/bash


if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi


cmd="$1"
```

```
found=""
for ext in "" .gz .bz2 .xz; do
    if [ -f "/usr/share/man/man1/${cmd}.1$ext" ]; then
        found="/usr/share/man/man1/${cmd}.1$ext"
        break
    fi
done

if [ -z "$found" ]; then
    echo "Справка для команды '$cmd' не найдена."
    exit 1
fi

case "$found" in
    *.gz)  zless "$found" ;;
    *.bz2) bzless "$found" ;;
    *.xz)  xzless "$found" ;;
    *)     less "$found" ;;
esac
```

3.3.2 Проверка работы

Скрипт успешно отображает справку для существующих команд (например, `ls`) и выдаёт сообщение об отсутствии для несуществующих. Результаты показаны на рис. 4 и 5.

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

cmd="$1"
man_dir="/usr/share/man/man1"
found=""
for ext in "" .gz .bz2 .xz .zst; do
    if [ -f "$man_dir/${cmd}.1$ext" ]; then
        found="$man_dir/${cmd}.1$ext"
        break
    fi
done

if [ -z "$found" ]; then
    echo "Справка для команды '$cmd' не найдена в $man_dir"
    exit 1
fi

case "$found" in
    *.gz)   zless "$found" ;;
    *.bz2)  bzless "$found" ;;
    *.xz)   xzless "$found" ;;
    *.zst)  zstdless "$found" ;;
    *)      less "$found" ;;
esac
```

Рисунок 3.4: Код task2.sh

```
ambessonov@fedora:~/lab14$ nano task2.sh
ambessonov@fedora:~/lab14$ chmod +x task2.sh
ambessonov@fedora:~/lab14$ ./task2.sh ls

[1]+  Остановлен    ./task2.sh ls
ambessonov@fedora:~/lab14$ ./task2.sh abcd
Справка для команды 'abcd' не найдена в /usr/share/man/man1
ambessonov@fedora:~/lab14$
```

Рисунок 3.5: Выполнение task2.sh для несуществующей команды

3.4 Задание 3. Генерация случайной последовательности букв латинского алфавита

3.4.1 Листинг скрипта task3.sh

```
#!/bin/bash

len=${1:-20}

if ! [[ "$len" =~ ^[1-9][0-9]*$ ]]; then
    echo "Ошибка: длина должна быть положительным целым числом"
    exit 1
fi

letters=(a b c d e f g h i j k l m n o p q r s t u v w x y z)
result=""

for ((i=0; i<len; i++)); do
    r=$(( RANDOM % 26 ))
    letter="${letters[$r]}"
    result="${result}${letter}"
done

echo "$result"
```

```
#!/bin/bash

len=$(1:-20)

if ! [[ "$len" =~ ^[1-9][0-9]*$ ]]; then
    echo "Ошибка: длина должна быть положительным целым числом"
    exit 1
fi

# Массив букв a-z
letters=(a b c d e f g h i j k l m n o p q r s t u v w x y z)

result=""
for ((i=0; i<len; i++)); do
    r=$(( RANDOM % 26 ))
    letter="${letters[$r]}"
    result="${result}${letter}"
done
echo "$result"
```

Рисунок 3.6: Код task3.sh

3.4.2 Проверка работы

Скрипт генерирует случайные строки заданной длины из строчных латинских букв. Примеры выполнения для длины 50 и 500 символов приведены на рис. 7.

```
ambessonov@fedora:~/lab14$ nano task3.sh
ambessonov@fedora:~/lab14$ chmod +x task3.sh
ambessonov@fedora:~/lab14$ ./task3.sh
ctqpkztydvvehdyyfktt
ambessonov@fedora:~/lab14$ ./task3.sh  50
bfhlndbgxylscdxeqfudoceibolnscnehzcjbmtramflexmmmn
ambessonov@fedora:~/lab14$ ./task3.sh  500
wxonitxotnibqsopwdnvrbmqmzjzoqcwfyhonncoyakcrvtuzzmfwar
hmxwurmanzjvqzuonoggsegvviqqsxamwqdrgbfkswukumgyumpxewt
snuilfguuhibmsrrbadckwptfpbgfgeinvltlcznsndwdcsazilgmnm
ambessonov@fedora:~/lab14$
```

Рисунок 3.7: Выполнение task3.sh с разной длиной

4 Выводы

В ходе лабораторной работы были освоены: - написание командных файлов на bash с использованием условных операторов, циклов, арифметических выражений; - синхронизация процессов с помощью файловых блокировок (`flock`) – реализован упрощённый семафор; - создание аналога команды `man` для просмотра справочных страниц; - генерация случайных последовательностей с использованием `$RANDOM` и преобразование чисел в символы.

Полученные навыки позволяют автоматизировать типовые задачи администрирования и разработки в среде UNIX.

5 Контрольные вопросы

5.1 Найдите синтаксическую ошибку в следующей строке:

```
while [$1 != "exit"]
```

Ответ:

- Отсутствуют пробелы между [и $1, а также между "exit" и].

- Переменная $1 должна быть заключена в двойные кавычки.

Исправленный вариант:

```
while [ "$1" != "exit" ]
```

5.2 Как объединить (конкатенация) несколько строк в одну?

Ответ:

В bash можно использовать прямое присваивание:

```
str="$line1$line2$line3"
```

Или через подстановку:

```
result="${result}${new_line}"
```

5.3 Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash?

Ответ:

seq генерирует последовательность чисел. Альтернативы в bash:

- Цикл `for ((i=1; i<=N; i++)); do echo $i; done`
- Оператор `{1..N}` (работает только с константами)
- Использование `eval echo {1..$N}` (потенциально опасно)
- Команда `printf '%d\n' {1..10}`

5.4 Какой результат даст вычисление выражения `$((10/3))`?

Ответ:

Целочисленное деление: **3**.

5.5 Укажите кратко основные отличия командной оболочки zsh от bash.

Ответ:

- В zsh индексация массивов начинается с 1 (в bash с 0).
- Более мощный глоббинг (`**/*.txt` рекурсивно).

- Нет необходимости экранировать фигурные скобки в регулярных выражениях.
- Встроенная поддержка ассоциативных массивов (в bash – с версии 4).
- Более настраиваемый промпт и автодополнение.

5.6 Проверьте, верен ли синтаксис данной конструкции:

```
for ((a=1; a <= LIMIT; a++))
```

Ответ:

Синтаксис неверен: не хватает закрывающей `))` и ключевого слова `do`.

Правильно:

```
for ((a=1; a <= LIMIT; a++)); do ...; done
```

5.7 Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки?

Ответ:

Сравнение с Python/C:

Преимущества bash:

- Простота запуска внешних команд и управления процессами.
- Встроенные средства для работы с файлами, конвейерами, перенаправлением ввода-вывода.
- Не требует компиляции, идеален для склеивания утилит.

Недостатки bash:

- Низкая производительность при вычислениях и обработке больших объёмов

данных.

- Слабая типизация, все переменные – строки.
- Сложный синтаксис для арифметики и работы с массивами.
- Много «подводных камней» (пробелы, кавычки, подстановки).
- Отсутствие модульной системы и нормальных структур данных.

Список литературы